Add or XOR

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t; cin >> t;
    while (t--) {
        int a, b, x, y;
        cin >> a >> b >> x >> y;
        const int MAXV = 200; // max state to consider
        vector<long long> dist(MAXV + 1, LLONG_MAX);
        dist[a] = 0;
        using pll = pair<long long, int>;
        priority_queue<pll, vector<pll>, greater<pll>> pq;
        pq.push({0, a});
        while (!pq.empty()) {
            auto [cost, u] = pq.top();
            pq.pop();
            if (cost > dist[u]) continue;
            if (u == b) break;
            // Operation 1: a + 1
            if (u + 1 <= MAXV && dist[u] + x < dist[u + 1]) {
                dist[u + 1] = dist[u] + x;
                pq.push({dist[u + 1], u + 1});
            }
            int v = u ^ 1;// Operation 2: a XOR 1
            if (v <= MAXV && dist[u] + y < dist[v]) {
                dist[v] = dist[u] + y;
                pq.push({dist[v], v});
            }
        }
        cout << (dist[b] == LLONG_MAX ? -1 : dist[b]) << "\n";
    }
}
```

Bitwise operations

```
 // Arithmetic operations

1 << i                                  // equals pow(2, i)
x << i                                  // equals x * pow(2,
     i)
x >> i                                  // equals floor(x /
    pow(2, i)) [for simplicity, assume x > 0]
x \& ((1 << i) - 1)                    // equals x \% pow(2, i)

// Bitmask construction

(1 << i)                                // bitmask with only i-
    th bit on
(1 << i) | (1 << j)                    // bitmask with only i-
    th and j-th bits on
\~0                                         // bitmask with
    all bits on (equal to-1)
\~(1 << i)                               // bitmask with all
    bits on except i-th bit off
-1 \^ (1 << i)                          // same as above

// Bit operations on variable x

x | (1 << i)                           // turn on the i-th bit
    of x
x \& \~(1 << i)                          // turn off the i-th
    bit of x
x \^ (1 << i)                          // toggle (flip) the i-
    th bit of x
(x >> i) \& 1                          // check if the i-th bit
     of x is on

// Tricks and built-in functions

x \& -x                                     // isolate the
    lowest bit that is on (LSB)
\__builtin\_popcountll(x)              // count number of bits
    that are on
```

0.1 BFS

```
enum State {READY, PROCESSING, VISITED};
pair<vector<int>, vector<int>> bfs(int n, vector<vector<int>>&
    adj, int source) {
    vector<State> state(n + 1, READY); // 1-based indexing
    queue<int> q;
    vector<int> dist(n + 1, -1);
    vector<int> parent(n + 1, 0);
    dist[source] = 0;
    state[source] = PROCESSING;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (state[v] == READY) {
                state[v] = PROCESSING;
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
        state[u] = VISITED;
    }
    return {dist, parent};
}
void trackPath(int source, int x, const vector<int>& parent) {
    if (parent[x] == 0 && x != source) {
        cout << "No path exists to node " << x << endl;
        return;
    }
    vector<int> path;
    while (x != 0) {
        path.push_back(x);
        x = parent[x];
    }
    reverse(path.begin(), path.end());
    cout << "Shortest path: ";
    for (int node : path) {
        cout << node << " ";
    }
    cout << endl;
}
int main() {
    int n, e, x;
    cin >> n >> e >> x;
    vector<vector<int>> adj(n + 1);

    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // For undirected graph
    }
    int source = 1;
    auto [dist, parent] = bfs(n, adj, source);
    trackPath(source, x, parent);
    return 0;
}
```

0.2 DFS

```
const int MAXN = 1e5 + 5;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent, tin, tout, dfs_order;
int timer = 0;

void dfs(int u, int par = -1) {
    visited[u] = true;
    parent[u] = par;
    tin[u] = ++timer;
    dfs_order.push_back(u);

    for (int v : adj[u]) {
        if (!visited[v]) dfs(v, u);
    }

    tout[u] = ++timer;
}

int main() {
    int n, m; // n = nodes, m = edges
    cin >> n >> m;

    adj.assign(n + 1, {});
    visited.assign(n + 1, false);
    parent.assign(n + 1, -1);
    tin.assign(n + 1, 0);
    tout.assign(n + 1, 0);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // remove this line if directed
    }

    for (int i = 1; i <= n; ++i)
```

```
        if (!visited[i])
            dfs(i);

    cout << "DFS Order: ";
    for (int u : dfs_order)
        cout << u << " ";
    cout << "\n";

    cout << "Discovery Times:\n";
    for (int i = 1; i <= n; ++i)
        cout << "Node " << i << ": tin = " << tin[i] << ", tout
            = " << tout[i] << "\n";

    return 0;
}
```

0.3 Lab 3: Journey to the moon

```
void bfs(int start, vector<vector<int>>& adj, vector<bool>&
     visited, vector<int>& component) {
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        component.push_back(u);
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<bool> visited(n, false);
    vector<int> component_sizes;
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            int size = 0;
            queue<int> q;
            q.push(i);
            visited[i] = true;
            while (!q.empty()) {
                int u = q.front(); q.pop();
                ++size;
                for (int v : adj[u]) {
                    if (!visited[v]) {
                        visited[v] = true;
                        q.push(v);
                    }
                }
            }
            component_sizes.push_back(size);
        }
    }

    long long total_pairs = 1LL * n * (n - 1) / 2,
         same_country_pairs = 0;
    for (int size : component_sizes)
        same_country_pairs += 1LL * size * (size - 1) / 2;

    cout << total_pairs - same_country_pairs << endl;
}
```

0.4 Lab 3: Rumor

```
int bfs(int start, const vector<vector<int>> &adj, const vector
     <int> &cost, vector<bool> &vis) {
    int minCost = cost[start];
    queue<int> q;
    q.push(start);
    vis[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!vis[v]) {
                vis[v] = true;
                minCost = min(minCost, cost[v]);
                q.push(v);
            }
        }
    }
    return minCost;
}

int main() {
    int n, m; cin >> n >> m;
    vector<int> cost(n);
    for (int &c : cost) cin >> c;
    vector<vector<int>> adj(n);
    for (int i = 0, u, v; i < m; ++i) {
        cin >> u >> v; --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<bool> vis(n, false);
    long long total = 0;
    for (int i = 0; i < n; ++i)
        if (!vis[i]) total += bfs(i, adj, cost, vis);
    cout << total << endl;
    return 0;
}
```

0.5 Lab 4: Bicycles (Dijkstra)

```
enum State {READY, PROCESSING, VISITED};

int dijkstra(int n, vector<vector<int>>& adj, vector<vector<int
     >>& weight, int source, vector<int> tmu) {
    vector<int> dist(n+1, INT_MAX), parent(n+1, -1);
    vector<State> state(n+1, READY);
    priority_queue<tuple<int, int>, vector<tuple<int, int>>,
         greater<>> pq;
    dist[source] = 0;
    state[source] = PROCESSING;
    pq.push({0, source});
    int currBike = 1;

    while (!pq.empty()) {
        int u = get<1>(pq.top());
        pq.pop();
        if (state[u] == VISITED) continue;
        if (tmu[currBike] > tmu[u]) currBike = u;

        for (int i = 0; i < adj[u].size(); i++) {
            int v = adj[u][i], w = weight[u][i];
            if (state[v] != VISITED && dist[u] + w * tmu[
                 currBike] < dist[v]) {
                dist[v] = dist[u] + w * tmu[currBike];
                parent[v] = u;
                pq.push({dist[v], v});
                state[v] = PROCESSING;
            }
        }
        state[u] = VISITED;
    }
    return dist[n];
}

int main() {
    int t; cin >> t;
    while (t--) {
        int n, m; cin >> n >> m;
        vector<vector<int>> adj(n+1), weight(n+1);
        vector<int> tmu(n+1);
        for (int i = 0; i < m; i++) {
            int u, v, w; cin >> u >> v >> w;
            adj[u].push_back(v); adj[v].push_back(u);
            weight[u].push_back(w); weight[v].push_back(w);
        }
        for (int i = 1; i <= n; i++) cin >> tmu[i];
        cout << dijkstra(n, adj, weight, 1, tmu) << endl;
    }
    return 0;
}
```

0.6 Lab 4: Izzhu and cities

```
typedef long long int lli;

enum State {READY, PROCESSING, VISITED};
```

```
int dijkstra(int n, vector<vector<int>>& adj, vector<vector<int
    >>& weight, int source, vector<vector<int>> routes) {
    vector<int> dist(n+1, INT_MAX), parent(n+1, -1);
    vector<State> state(n+1, READY);
    priority_queue<tuple<int,int>, vector<tuple<int,int>>,
        greater<>> pq;
    dist[source] = 0;
    state[source] = PROCESSING;
    pq.push({0, source});
    int rmRoute = 0;

    while (!pq.empty()) {
        int u = get<1>(pq.top());
        pq.pop();
        if (state[u] == VISITED) continue;

        for (int i = 0; i < adj[u].size(); i++) {
            int v = adj[u][i], w = weight[u][i];
            if (state[v] != VISITED && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.push({dist[v], v});
                state[v] = PROCESSING;
            }
        }
        state[u] = VISITED;
    }

    for (int i = 2; i <= n; i++) {
        if (routes[i].empty()) continue;
        bool oneKept = false;
        for (int y : routes[i]) {
            if (y == dist[i] && !oneKept) oneKept = true;
            else rmRoute++;
        }
    }
    return rmRoute;
}

int main() {
    int n, m, k; cin >> n >> m >> k;
    vector<vector<int>> adj(n+1), weight(n+1), routes(n+1);
    for (int i = 0; i < m; i++) {
        int u, v, x; cin >> u >> v >> x;
        adj[u].push_back(v);
        adj[v].push_back(u);
        weight[u].push_back(x);
        weight[v].push_back(x);
    }
    for (int i = 0; i < k; i++) {
        int s, y; cin >> s >> y;
        adj[1].push_back(s);
        adj[s].push_back(1);
        weight[1].push_back(y);
        weight[s].push_back(y);
        routes[s].push_back(y);
    }
    cout << dijkstra(n, adj, weight, 1, routes) << "\n";
}
```

0.7 Lab 4: Graph and Graph

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t; cin >> t;
    while (t--) {
        int n, s1, s2; cin >> n >> s1 >> s2;
        int m1; cin >> m1;
        vector<vector<int>> g1(n + 1), g2(n + 1);
        for (int i = 0; i < m1; i++) {
            int a, b; cin >> a >> b;
            g1[a].push_back(b);
            g1[b].push_back(a);
        }
        int m2; cin >> m2;
        for (int i = 0; i < m2; i++) {
            int c, d; cin >> c >> d;
            g2[c].push_back(d);
            g2[d].push_back(c);
        }

        // We want to find a zero-cost infinite cycle reachable
            from (s1, s2)
        // Zero cost means moving simultaneously to same vertex
            in both graphs.
        // So edges in product graph where u1 == u2.
        // BFS/DFS on product graph with edges (v1,v2) -> (u,u)
            if u adjacent to v1 in G1 and u adjacent to v2 in
            G2
        // Use BFS/DFS to find reachable nodes, and detect cycle
             in zero-cost product graph
        vector<vector<bool>> visited(n+1, vector<bool>(n+1,
            false));
        vector<vector<bool>> in_stack(n+1, vector<bool>(n+1,
            false));
        bool has_cycle = false;

        function<void(int,int)> dfs = [&](int v1,int v2) {
            if (has_cycle) return;
            visited[v1][v2] = true;
            in_stack[v1][v2] = true;
            // For each u adjacent to v1 in g1 and also adjacent
                 to v2 in g2
            // But we want u adjacent to v1 and u adjacent to v2
                 simultaneously
            // To speed up adjacency check, use sets for g2
            // Let's build sets first
        };
        // Build sets for g2 adjacency to speed up checks
        vector<unordered_set<int>> g2set(n+1);
        for (int i = 1; i <= n; i++) {
            for (int u : g2[i]) g2set[i].insert(u);
        }
        function<void(int,int)> dfs2 = [&](int v1,int v2) {
            if (has_cycle) return;
            visited[v1][v2] = true;
            in_stack[v1][v2] = true;
            for (int u : g1[v1]) {
                if (g2set[v2].count(u)) { // zero cost move
                     possible
                    if (!visited[u][u]) {
                        dfs2(u,u);
                    } else if (in_stack[u][u]) {
                        has_cycle = true;
                        return;
                    }
                }
            }
            in_stack[v1][v2] = false;
        };
        dfs2(s1, s2);
        if (has_cycle) cout << 0 << "\n";
        else cout << -1 << "\n";
    }
    return 0;
}
```

0.8 Lab 5: Puran Dhaka (Floyd-Warshall)

```
typedef long long ll;
void floydWarshall(int n,vector<vector<ll>>& dist){
    for(int k=1;k<=n;k++){
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                if(dist[i][k]!=INT_MAX && dist[k][j]!=INT_MAX)
                    dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][
                        j]);
            }
        }
    }
}
int main(){
    int n,m,q;cin>>n>>m>>q;
    vector<vector<ll>> dist(n+1,vector<ll>(n+1,INT_MAX));
    for(int i=1;i<=n;i++) dist[i][i]=0;
    for(int i=0;i<m;i++){
        int a,b; ll c; cin>>a>>b>>c;
        dist[a][b]=min(dist[a][b],c);
        dist[b][a]=min(dist[b][a],c);
    }
    floydWarshall(n,dist);
    while(q--){
        int a,b; cin>>a>>b;
        if(dist[a][b]==INT_MAX) cout<<-1<<endl;
        else cout<<dist[a][b]<<endl;
    }
}
```

0.9 lab 5: Dekisugi (Bellman-Ford)

```cpp
#include <bits/stdc++.h>
using namespace std;

void printNegativeCycle(int start, vector<int>& parent) {
    int n = parent.size() - 1;
    vector<bool> visited(n + 1, false);
    for (int i = 0; i < n; ++i) start = parent[start]; // move
         n steps to enter cycle
    int cur = start;
    vector<int> cycle;
    do {
        cycle.push_back(cur);
        cur = parent[cur];
    } while (cur != start);
    cycle.push_back(start);
    reverse(cycle.begin(), cycle.end());
    cout << "YES\n";
    for (int x : cycle) cout << x << " ";
    cout << "\n";
}

bool bellmanFord(int n, vector<vector<int>>& edges, int src,
     vector<int>& parent) {
    vector<long long> dist(n + 1, 1e18);
    dist[src] = 0;
    parent.assign(n + 1, -1); // Initialize parent array for
         path tracking
    int updatedNode = -1;
    for (int i = 1; i <= n; ++i) {
        updatedNode = -1;
        for (auto& e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u; // Track the parent for path
                     reconstruction
                updatedNode = v;
            }
        }
    }
    if (updatedNode == -1) {
        cout << "NO\n";
        return false;
    } else {
        printNegativeCycle(updatedNode, parent);
        return true;
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> edges(m, vector<int>(3));
    for (int i = 0; i < m; ++i) cin >> edges[i][0] >> edges[i
         ][1] >> edges[i][2];
    vector<int> parent; // Parent array will be filled in
         bellmanFord
    bellmanFord(n, edges, 1, parent);
    return 0;
}
```

0.10 Lab 5: Signal Hill (DFS)

```cpp
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;
const int MAXN = 1005;
struct Beacon {
    int x, y, r;
};
int n, q;
vector<Beacon> beacons;
vector<int> adj[MAXN];
vector<bool> visited;
// Euclidean distance squared
bool canReach(const Beacon& from, const Beacon& to) {
    long long dx = from.x - to.x;
    long long dy = from.y - to.y;
    long long distSq = dx * dx + dy * dy;
    return distSq <= 1LL * from.r * from.r;
}
void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}
int main() {
    cin >> n >> q;
    beacons.resize(n + 1); // 1-based indexing
    for (int i = 1; i <= n; ++i) {
        int x, y, r;
        cin >> x >> y >> r;
        beacons[i] = {x, y, r};
    }
    // Build directed graph
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (i != j && canReach(beacons[i], beacons[j])) {
                adj[i].push_back(j);
            }
        }
    }
    // Answer queries
    while (q--) {
        int a, b;
        cin >> a >> b;
        visited.assign(n + 1, false);
        dfs(a);
        cout << (visited[b] ? "YES" : "NO") << '\n';
    }
    return 0;
}
```

0.11 Lab 5: Roads in Berland

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Floyd Warshall to compute all pairs shortest paths initially
void floydWarshall(int n, vector<vector<long long>>& dist){
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(dist[i][k]+dist[k][j]<dist[i][j])
                    dist[i][j]=dist[i][k]+dist[k][j];
}

// Update dist matrix and sumDist after adding a new edge (a,b,
     c)
long long updateWithNewEdge(int n, vector<vector<long long>>&
     dist, int a, int b, long long c, long long sumDist){
    if(c>=dist[a][b]) return sumDist;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            long long newDist=min(dist[i][a]+c+dist[b][j], dist[
                 i][b]+c+dist[a][j]);
            if(newDist<dist[i][j]){
                sumDist-=(dist[i][j]-newDist);
                dist[i][j]=newDist;
                dist[j][i]=newDist;
            }
        }
    }
    return sumDist;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin>>n;
    vector<vector<long long>> dist(n,vector<long long>(n));
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin>>dist[i][j];
    floydWarshall(n,dist);
    long long sumDist=0;
    for(int i=0;i<n;i++)
        for(int j=i+1;j<n;j++)
            sumDist+=dist[i][j];
    int k; cin>>k;
    while(k--){
        int a,b; long long c;
        cin>>a>>b>>c;
        a--; b--;
        sumDist=updateWithNewEdge(n,dist,a,b,c,sumDist);
        cout<<sumDist<<" ";
    }
    return 0;
}
```

0.12 Lab 5: High Score (Bellman and BFS)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int a,b;
    long long x;
};

int n,m;
vector<Edge> edges;
vector<vector<int>> adj, rev; // adjacency and reverse
     adjacency

bool reachableFromStart[2505], reachableToEnd[2505];

void dfsReachFromStart(int u) {
    reachableFromStart[u] = true;
    for(int v : adj[u]) if(!reachableFromStart[v])
         dfsReachFromStart(v);
}
void dfsReachToEnd(int u) {
    reachableToEnd[u] = true;
    for(int v : rev[u]) if(!reachableToEnd[v]) dfsReachToEnd(v)
         ;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;
    edges.resize(m);
    adj.assign(n+1, vector<int>());
    rev.assign(n+1, vector<int>());

    for(int i=0;i<m;i++){
        cin >> edges[i].a >> edges[i].b >> edges[i].x;
        adj[edges[i].a].push_back(edges[i].b);
        rev[edges[i].b].push_back(edges[i].a);
    }

    // Find which nodes reachable from start and which can
         reach end
    dfsReachFromStart(1);
    dfsReachToEnd(n);

    // Initialize distance array with very small values (long
         long min)
    vector<long long> dist(n+1, LLONG_MIN);
    dist[1] = 0;

    // Bellman-Ford for longest path (relax edges up to n-1
         times)
    for(int i=0;i<n-1;i++){
        bool updated = false;
        for(auto &e : edges){
            if(dist[e.a] == LLONG_MIN) continue;
            if(dist[e.a] + e.x > dist[e.b]){
                dist[e.b] = dist[e.a] + e.x;
                updated = true;
            }
        }
        if(!updated) break;
    }

    // Detect positive cycles on paths from 1 to n
    // Relax one more time and check if distance can still
         improve
    for(auto &e : edges){
        if(dist[e.a] == LLONG_MIN) continue;
        if(dist[e.a] + e.x > dist[e.b]){
            // Check if this cycle is on a path from 1 to n
            // That means node e.b reachable from 1 and can
                 reach n
            if(reachableFromStart[e.b] && reachableToEnd[e.b]){
                cout << -1 << "\n";
                return 0;
            }
        }
    }

    // If no positive cycle detected on the path 1->n, print
         max dist to n
    cout << dist[n] << "\n";

    return 0;
}
```

0.13 Lab 5: Edge Deletion

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long INF = 1e15;
int main() {
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, int>> edges(m);
    vector<vector<long long>> dist(n + 1, vector<long long>(n +
         1, INF));
    for (int i = 1; i <= n; ++i) dist[i][i] = 0;

    for (int i = 0; i < m; ++i) {
        int a, b, c;
        cin >> a >> b >> c;
        edges[i] = {a, b, c};
        dist[a][b] = min(dist[a][b], 1LL * c);
        dist[b][a] = min(dist[b][a], 1LL * c);
    }
    // Floyd-Warshall
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k
                     ][j]);
    int deletable = 0;
    for (auto [u, v, w] : edges) {
        bool necessary = false;
        for (int i = 1; i <= n && !necessary; ++i)
            for (int j = 1; j <= n && !necessary; ++j)
                if (dist[i][j] == dist[i][u] + w + dist[v][j] ||
                    dist[i][j] == dist[i][v] + w + dist[u][j])
                    necessary = true;
        if (!necessary) deletable++;
    }
    cout << deletable << "\n";
    return 0;
}
```

0.14 Lab 5: Flight Discount

```cpp
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const ll INF=1e18;

vector<vector<ll>> dijkstra(int n,vector<vector<pair<int,ll>>>&
     adj){
  // dist[u][0] = min cost to reach u without using coupon
  // dist[u][1] = min cost to reach u after using coupon
  vector<vector<ll>> dist(n+1,vector<ll>(2,INF));
  priority_queue<tuple<ll,int,int>,vector<tuple<ll,int,int>>,
       greater<>> pq;
  dist[1][0]=0;
  pq.push({0,1,0});
  while(!pq.empty()){
    auto [d,u,state]=pq.top(); pq.pop();
    if(d>dist[u][state]) continue;
    for(auto [v,c]:adj[u]){
      // normal edge
      if(dist[v][state]>d+c){
        dist[v][state]=d+c;
        pq.push({dist[v][state],v,state});
      }
      // use coupon if not used yet
      if(state==0 && dist[v][1]>d+c/2){
        dist[v][1]=d+c/2;
        pq.push({dist[v][1],v,1});
      }
    }
  }
  return dist;
}

int main(){
  int n,m;
  cin>>n>>m;
  vector<vector<pair<int,ll>>> adj(n+1);
  for(int i=0;i<m;i++){
    int a,b;
    ll c;
    cin>>a>>b>>c;
    adj[a].push_back({b,c});
  }
  vector<vector<ll>> dist=dijkstra(n,adj);
```

```
  cout<<dist[n][1]<<endl;
}
```

0.15 Lab 5: Greg and Graph

```
typedef long long ll;
const int N=505;
ll dist[N][N];
bool added[N];
vector<int> order;
vector<ll> res;
// Run Floyd-Warshall update with newly added node k
void floyd(int k,int n){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][j]);
        }
    }
}
int main(){
    ios::sync_with_stdio(0);cin.tie(0);
    int n;cin>>n;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++) cin>>dist[i][j];
    }
    order.resize(n);
    for(int i=0;i<n;i++) cin>>order[i];
    reverse(order.begin(),order.end()); // simulate vertex
         addition
    for(int idx=0;idx<n;idx++){
        int k=order[idx];
        added[k]=true;
        floyd(k,n);
        ll sum=0;
        for(int i=1;i<=n;i++){
            if(!added[i]) continue;
            for(int j=1;j<=n;j++){
                if(!added[j]) continue;
                sum+=dist[i][j];
            }
        }
        res.push_back(sum);
    }
    reverse(res.begin(),res.end()); // get output in original
         order
    for(ll val:res) cout<<val<<" ";
    return 0;
}
```

0.16 Prim and Kruskal

```
#include <bits/stdc++.h>
using namespace std;
struct Edge {
    int u,v;
    long long w;
    bool operator<(const Edge &other) const {
        return w < other.w;
    }
};
const int MAXN=100005;
int parent[MAXN],rank_arr[MAXN];
// DSU find
int find(int a) {
    if(parent[a]==a) return a;
    return parent[a]=find(parent[a]);
}
// Your unite function exactly as given
bool unite(int a,int b) {
    a=find(a);
    b=find(b);
    if(a==b) return false; // already in same set
    if(rank_arr[a]<rank_arr[b]) swap(a,b);
    parent[b]=a;
    if(rank_arr[a]==rank_arr[b]) rank_arr[a]++;
    return true;
}
// Kruskal MST
long long kruskal(int n,vector<Edge> &edges) {
    for(int i=1;i<=n;i++) {
        parent[i]=i;
        rank_arr[i]=0;
    }
    sort(edges.begin(),edges.end());
    long long cost=0;
    for(auto &e:edges) {
        if(unite(e.u,e.v)) cost+=e.w;
    }
    return cost;
}
// Prim MST (unchanged)
long long prim(int n,vector<vector<pair<int,long long>>> &adj)
     {
    vector<bool> visited(n+1,false);
    vector<long long> dist(n+1,LLONG_MAX);
    dist[1]=0;
    long long cost=0;
    priority_queue<pair<long long,int>,vector<pair<long long,
         int>>,greater<>> pq;
    pq.push({0,1});
    while(!pq.empty()) {
        auto [d,u]=pq.top();pq.pop();
        if(visited[u]) continue;
        visited[u]=true;
        cost+=d;
        for(auto &[v,w]:adj[u]) {
            if(!visited[v]&&w<dist[v]) {
                dist[v]=w;
                pq.push({w,v});
            }
        }
    }
    return cost;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;cin>>n>>m;
    vector<Edge> edges(m);
    for(int i=0;i<m;i++) cin>>edges[i].u>>edges[i].v>>edges[i].
         w;
    // Kruskal MST
    long long ans_kruskal=kruskal(n,edges);
    cout<<"Kruskal MST cost: "<<ans_kruskal<<"\n";
    // Prim MST (uncomment to use)
    /*
    vector<vector<pair<int,long long>>> adj(n+1);
    for(auto &e:edges) {
        adj[e.u].push_back({e.v,e.w});
        adj[e.v].push_back({e.u,e.w});
    }
    long long ans_prim=prim(n,adj);
    cout<<"Prim MST cost: "<<ans_prim<<"\n";
    */
    return 0;
}
```